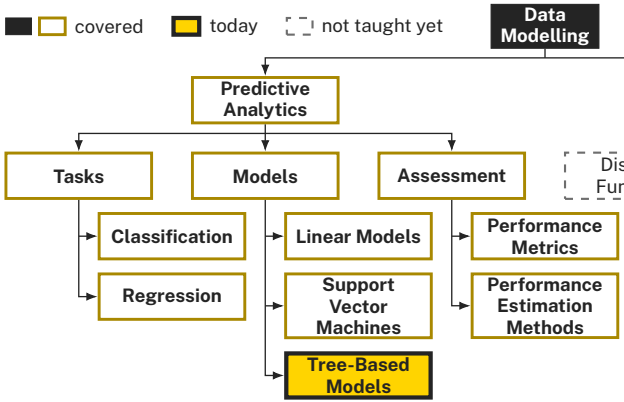
DALHOUSIE UNIVERSITY

Decision Trees

Recursive partitioning, impurity, and reading a fitted tree

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Where Tree-Based Models Sit



Today we meet decision trees.
After kNN, LDA and the SVM, a model you can read as rules.

A Model You Can Read

Every classifier so far gives a label. Ask it *why*, and the answers differ.

kNN

"These five neighbours voted so." There is no rule to read.

LDA

A weighted sum of the predictors crossed a threshold.

SVM

The case was similar to some support vectors. A radial kernel has nothing else to read.

Tree

Two or three questions about the case, in its own units.

A tree is a short list of yes-or-no questions, learned from the data and evaluated with the usual tools.

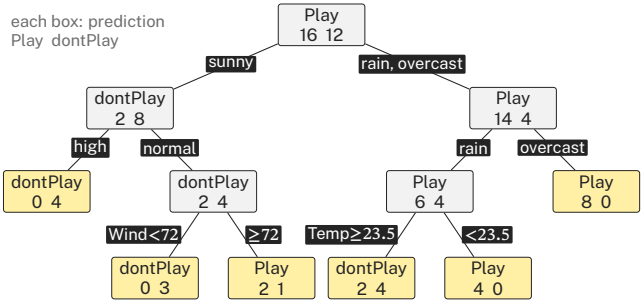
The Weather Data

Weather	Temp	Humidity	Wind	Decision	Weather	Temp	Humidity	Wind	Decision
rain	26	high	15	dontPlay	rain	29	high	79	dontPlay
rain	35	normal	102	dontPlay	rain	26	normal	75	dontPlay
overcast	27	high	99	Play	overcast	30	high	108	Play
overcast	26	normal	97	Play	overcast	30	normal	16	Play
rain	12	high	120	Play	rain	33	high	96	Play
overcast	21	normal	74	Play	overcast	30	normal	13	Play
sunny	30	normal	89	dontPlay	sunny	32	normal	55	dontPlay
sunny	19	high	111	dontPlay	sunny	11	high	108	dontPlay
sunny	14	normal	81	Play	sunny	33	normal	103	Play
overcast	10	normal	70	Play	overcast	14	normal	32	Play
rain	11	normal	95	Play	rain	28	normal	44	Play
rain	15	high	94	Play	rain	21	high	84	Play
sunny	19	high	41	dontPlay	sunny	29	high	105	dontPlay
sunny	35	normal	38	dontPlay	sunny	15	normal	63	dontPlay

28 days, four predictors, two of them categories and two numbers, and one target, `Decision`, with **16 Play and 12 dontPlay**. Every idea today is shown on these rows.

After Quinlan's weather data (1986).

The Tree We Will Grow



- The **root** holds all 28 days, 16 Play and 12 dontPlay.
- Each **internal node** asks one question and sends every day down exactly one branch.
- Each **leaf** asks nothing. It predicts its majority class, and its counts say how sure it is.
- Six leaves, three questions deep, 3 of 28 days wrong.

By the end of today you can grow it by hand, read it, and say why it stopped where it did.

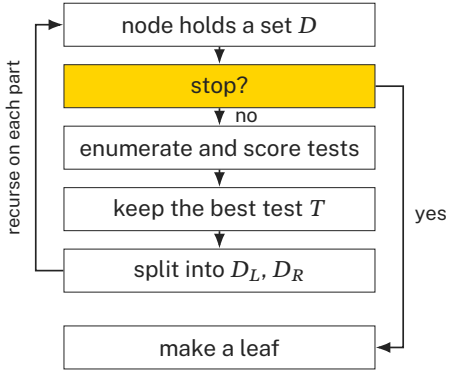
Growing a Tree

Ask the one question that best separates the classes. Then ask again inside each answer.

The Algorithm in Plain Words

1. Start with every case in one box.
2. If the box is pure, too small or deep enough, make it a leaf that predicts its majority class.
3. Otherwise list every question each predictor allows, score each one, and keep the best.
4. Split the box in two and repeat inside each half.

Greedy, the best question now, never revisited.
Recursive, each half is the same problem, smaller.
The next slides fill in step 3.

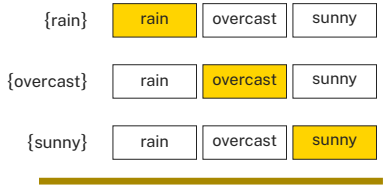


Which Questions Can a Tree Ask?

DALHOUSIE UNIVERSITY

A category

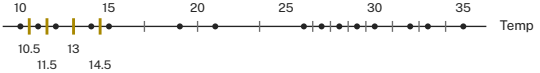
Send some levels left and the rest right. `Weather` has three levels, so three questions. `Humidity` has two, so one.



gold = levels sent left; the complement goes right

A number

Sort the values and try a cut halfway between each pair of neighbours. `Temp` has 15 distinct values, so 14 cuts.



Only the order of the values decides who falls on which side, so a tree needs no scaling, unlike *k*NN and the SVM.

How Mixed Is a Box?

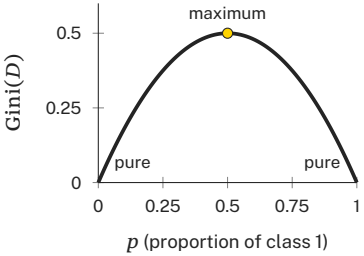
A good question leaves each answer purer than the box it came from.

Measuring Mixing With Gini

Draw two cases at random from a box, putting the first back. **Gini impurity** is the chance that they belong to different classes.

$$\text{Gini} = 1 - (\text{share of Play})^2 - (\text{share of dontPlay})^2$$

- The 28 days. Both draws Play, $(\frac{16}{28})^2 = 0.327$. Both dontPlay, $(\frac{12}{28})^2 = 0.184$. They agree with chance 0.510, so Gini is $1 - 0.510 = 0.490$.
- A pure box scores 0. A 50–50 box scores 0.5, the most mixed two classes can be.



With more classes, subtract one square per class, $1 - \sum_c p_c^2$, where p_c is the share of class c.

Scoring a Question

A question is worth the mixing it removes. Average the two sides' Gini, weighted by how many cases each holds, and subtract it from the parent's.

$$\text{removed} = \text{Gini}(D) - \frac{n_L}{n}\,\text{Gini}(D_L) - \frac{n_R}{n}\,\text{Gini}(D_R)$$

D is the box of n cases, and D_L, D_R are the n_L and n_R cases each answer sends.

Is it a rain day?	days	Play, dontPlay	Gini
yes	10	6, 4	$1 - 0.6^2 - 0.4^2 = 0.480$
no	18	10, 8	$1 - (\frac{10}{18})^2 - (\frac{8}{18})^2 = 0.494$
weighted average			$\frac{10}{28}(0.480) + \frac{18}{28}(0.494) = 0.489$
removed			$0.490 - 0.489 = \mathbf{0.001}$

Both sides are about as mixed as the parent, so asking about rain buys almost nothing.

A Pure Side Is Not Enough

The same arithmetic for all three `Weather` questions.

	answer yes		answer no		
Question	days (Play, dontPlay)	Gini	days (Play, dontPlay)	Gini	Gini after, removed
Is it rain?	10 (6, 4)	0.480	18 (10, 8)	0.494	0.489, 0.001
Is it overcast?	8 (8, 0)	0	20 (8, 12)	0.480	0.343, 0.147
Is it sunny?	10 (2, 8)	0.320	18 (14, 4)	0.346	**0.337, 0.153**

Overcast gives a pure side, 8 days, all Play, Gini 0. The other 20 days, 8 to 12, score $1 - 0.4^2 - 0.6^2 = 0.480$, so $\frac{8}{28}(0) + \frac{20}{28}(0.480) = 0.343$ is left.

Sunny leaves both sides mixed, $1 - 0.2^2 - 0.8^2 = 0.320$ on 10 days and 0.346 on 18. $\frac{10}{28}(0.320) + \frac{18}{28}(0.346) = 0.337$ is left. That is lower, so sunny wins.

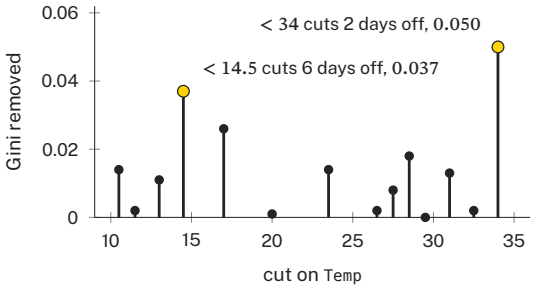
A pure side counts only for the cases it holds, and 8 of 28 days is not enough here.

Scoring Every Cut on Temp

A cut is a question like any other. `Temp` < 14.5 sends the 6 coldest days left, 10, 11, 11, 12, 14 and 14 degrees.

	days	Play, dontPlay
below 14.5	6	5, 1
14.5 or more	22	11, 11

$$\text{below} = 1 - (\tfrac{5}{6})^2 - (\tfrac{1}{6})^2 = 0.278$$
$$\text{14.5 or more} = 1 - 0.5^2 - 0.5^2 = 0.500$$
$$\text{after} = \tfrac{6}{28}(0.278) + \tfrac{22}{28}(0.500) = 0.452$$
$$\text{removed} = 0.490 - 0.452 = \mathbf{0.037}$$



Repeat this for all 14 cuts. The best, < 34, removes 0.050, but only by cutting off the two 35-degree days, both dontPlay. A two-day side is another small pure side.

Gini in a Few Lines

DALHOUSIE UNIVERSITY

R

```r
gini <- function(y) {
  p <- prop.table(table(y))
  1 - sum(p^2)
}
gain <- function(impurity, y, keep) {
  n <- length(y)   # mixing removed by the split
  left <- sum(keep) / n * impurity(y[keep])
  right <- sum(!keep) / n * impurity(y[!keep])
  impurity(y) - left - right
}
y <- w$Decision; wx <- w$Weather
gini(y)                              # 0.4897959
gain(gini, y, wx == "rain")          # 0.0009070295
gain(gini, y, wx == "overcast")      # 0.1469388
gain(gini, y, wx == "sunny")         # 0.153288
```

Python

```python
def gini(y):
    p = y.value_counts(normalize=True).to_numpy()
    return 1 - (p ** 2).sum()

def gain(impurity, y, keep):  # mixing removed
    n = len(y)
    left = keep.sum() / n * impurity(y[keep])
    right = (~keep).sum() / n * impurity(y[~keep])
    return impurity(y) - left - right

y, wx = w["Decision"], w["Weather"]
gini(y)                              # 0.4897959
gain(gini, y, wx == "rain")          # 0.0009070
gain(gini, y, wx == "overcast")      # 0.1469388
gain(gini, y, wx == "sunny")         # 0.1532880
```

`gain` takes the impurity as an argument, so the same lines score a split with entropy later.

Every Question on Every Predictor

	all 28 days			the 18 not sunny		
Predictor	tried	best question	removed	tried	best question	removed
Weather	3	= sunny	**0.153**	1	= overcast	**0.079**
Temp	14	< 34	0.050	12	< 34	0.071
Humidity	1	= high	0.008	1	= high	0.001
Wind	26	< 80	0.021	17	< 81.5	0.025

At the root the program scores all 44 questions and keeps the best, `Weather = sunny`, which removes 0.153. The best number cut, `Temp < 34`, removes only 0.050.

Inside the 18 days that are not sunny the whole search runs again on those days alone. `Weather = overcast` now wins with 0.079. That is the recursion, the same step on a smaller box.

The Search as Code

R

```r
questions <- function(x) {  # every yes-or-no question
  if (is.numeric(x)) {      # midpoints of values
    v <- sort(unique(x))
    c <- (head(v, -1) + tail(v, -1)) / 2
    return(setNames(lapply(c, \(t) x < t), paste("<", c)))
  }
  lv <- sort(unique(x))     # one level vs the rest
  if (length(lv) == 2) lv <- lv[1]
  setNames(lapply(lv, \(v) x == v), paste("=", lv))
}
best <- function(d) for (col in names(d)[1:4]) {
  s <- sapply(questions(d[[col]]),  # every question
              function(k) gain(gini, d$Decision, k))
  cat(col, length(s), names(which.max(s)),
      round(max(s), 3), "\n")      # and the best
}
best(w)                           # all 28 days
best(w[w$Weather != "sunny", ])   # the 18 others
```

Python

```python
def questions(x):  # every yes-or-no question on x
    if pd.api.types.is_numeric_dtype(x):
        v = np.unique(x)   # midpoints of sorted values
        cuts = (v[:-1] + v[1:]) / 2
        return {f"< {c:g}": x < c for c in cuts}
    lv = sorted(x.unique())  # one level vs the rest
    lv = lv[:1] if len(lv) == 2 else lv
    return {"= " + v: x == v for v in lv}

def best(d):             # best question per predictor
    for col in d.columns[:4]:
        s = {q: gain(gini, d["Decision"], k)
             for q, k in questions(d[col]).items()}
        q = max(s, key=s.get)
        print(col, len(s), q, round(s[q], 3))

best(w)                           # all 28 days
best(w[w["Weather"] != "sunny"])  # the 18 others
```

Each printed line is one row of the table on the previous slide.

Every Node, Its Question, and Why It Stops

The search reruns at each node. A side must keep 3 days or more, and no path asks over 3 questions.

Node	days (Play, dontPlay)	Gini	best question	removed	then
all days	28 (16, 12)	0.490	Weather = sunny	0.153	split
sunny	10 (2, 8)	0.320	Humidity = high*	0.053	split
high	4 (0, 4)	0			stop, pure
normal	6 (2, 4)	0.444	Wind < 72	0.222	split
Wind < 72	3 (0, 3)	0			stop, pure
Wind ≥ 72	3 (2, 1)	0.444			stop, depth 3
not sunny	18 (14, 4)	0.346	Weather = overcast	0.079	split
overcast	8 (8, 0)	0			stop, pure
rain	10 (6, 4)	0.480	Temp < 23.5	0.213	split
Temp < 23.5	4 (4, 0)	0			stop, pure
Temp ≥ 23.5	6 (2, 4)	0.444			stop, depth 3

*A tie. `Wind` < 72 also splits the sunny days 4 (0, 4) and 6 (2, 4) and removes the same 0.053. `rpart` keeps the predictor named first. Name `Wind` first and the tree keeps its 3 errors but loses `Humidity`.

A Second Way to Measure Mixing

Entropy measures, in bits, how uncertain a case's class is. A 50–50 box is 1 bit, the answer to one fair yes-or-no question. A pure box is 0.

$$H = -\sum_c p_c \log_2 p_c$$

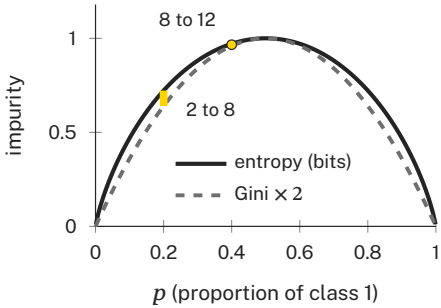
- The 28 days, $-\frac{16}{28}\log_2\frac{16}{28} - \frac{12}{28}\log_2\frac{12}{28} = 0.985$ bits.
- Overcast leaves 8 pure days and 20 at 8 to 12, $-0.4\log_2 0.4 - 0.6\log_2 0.6 = 0.971$ bits, and removes $0.985 - \frac{20}{28}(0.971) = 0.292$.
- Sunny leaves 0.722 and 0.764 bits and removes 0.236.

The entropy a question removes is its **information gain**. It is scored exactly like Gini.

	removed at the root	
question	Gini	entropy
overcast	0.147	**0.292**
sunny	**0.153**	0.236

The two measures pick different roots.

Why the Two Measures Disagree



Double Gini, so both curves are 1 at 50–50.

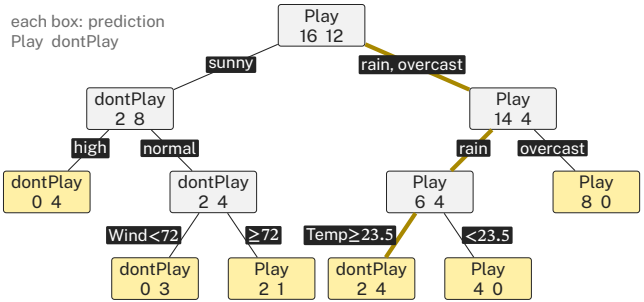
- Near 50–50 they agree. Overcast's mixed side, 8 to 12, has entropy 0.971 and doubled Gini 0.960.
- Toward the ends entropy is higher. Sunny's sides, 2 to 8 and 14 to 4, have 0.722 and 0.764 against 0.640 and 0.691.
- So entropy charges sunny's lopsided sides more. Overcast's pure side costs nothing under either.

Ask for entropy with `parms = list(split = "information")` in `rpart` or `criterion="entropy"` in scikit-learn, and both put overcast at the root. Gini is the default in both. Report which you used.

Reading a Fitted Tree

A prediction is a path from the root to one leaf.

Follow One New Day Down



A new day, **rain, Temp 25, normal humidity, Wind 40**.

1. Sunny? No, go right.
2. Rain or overcast? Rain.
3. Temp at least 23.5? Yes, 25.

The leaf holds 2 Play and 4 dontPlay. The tree predicts **dontPlay**, with class shares $\frac{4}{6} = 0.67$ and $\frac{2}{6} = 0.33$.

Humidity and Wind were never asked. Only the questions on the path matter for this day.

The Running Tree in R and Python

DALHOUSIE UNIVERSITY

R

```r
library(rpart)
fit <- rpart(Decision ~ Weather + Temp +
    Humidity + Wind, data = w, method = "class",
  control = rpart.control(cp = 0, maxdepth = 3,
    minsplit = 6, minbucket = 3))
print(fit, digits = 3)
```

Python

```python
X4 = pd.get_dummies(w.drop(columns="Decision"),
    dtype=int)              # 0/1 per level
fit = tree.DecisionTreeClassifier(
    criterion="gini", max_depth=3,
    min_samples_split=6, min_samples_leaf=3,
    random_state=0).fit(X4, w["Decision"])
print(tree.export_text(fit, spacing=2,
    decimals=1, show_weights=True,
    feature_names=list(X4.columns)))
```

The same Gini, depth at most 3, no leaf under 3 days. `rpart` takes `Weather` as a category. scikit-learn needs numbers, so `get_dummies` turns each level into a 0 or 1 column and `Weather_sunny <= 0.5` means not sunny.

R's Printout, One Line per Node

```
# node), split, n, loss, yval, (yprob)
#  1) root 28 12 Play (0.429 0.571)
#    2) Weather=sunny 10 2 dontPlay (0.800 0.200)
#      4) Humidity=high 4 0 dontPlay (1.000 0.000) *
#      5) Humidity=normal 6 2 dontPlay (0.667 0.333)
#       10) Wind< 72 3 0 dontPlay (1.000 0.000) *
#       11) Wind>=72 3 1 Play (0.333 0.667) *
#    3) Weather=overcast,rain 18 4 Play (0.222 0.778)
#      6) Weather=rain 10 4 Play (0.400 0.600)
#       12) Temp>=23.5 6 2 dontPlay (0.667 0.333) *
#       13) Temp< 23.5 4 0 Play (0.000 1.000) *
#      7) Weather=overcast 8 0 Play (0.000 1.000) *
```

Read node 6 left to right.

`6)`	node 6, its children are 12 and 13
`Weather=rain`	the answer that led here
`10`	days that reached it
`4`	days it gets wrong
`Play`	its prediction, the majority
`(0.400 0.600)`	shares of dontPlay, Play

A star marks a leaf. Indentation is depth. Node 12 is a leaf of 6 days, 2 of them wrong, predicting dontPlay with shares 0.667 and 0.333, the new day's leaf.

Python's Printout, One Line per Branch

```
# |-- Weather_sunny <= 0.5
# |   |-- Weather_rain <= 0.5
# |   |   |-- weights: [8.0, 0.0] class: Play
# |   |-- Weather_rain >  0.5
# |   |   |-- Temp <= 23.5
# |   |   |   |-- weights: [4.0, 0.0] class: Play
# |   |   |-- Temp >  23.5
# |   |   |   |-- weights: [2.0, 4.0] class: dontPlay
# |-- Weather_sunny >  0.5
# |   |-- Humidity_high <= 0.5
# |   |   |-- Wind <= 72.0
# |   |   |   |-- weights: [0.0, 3.0] class: dontPlay
# |   |   |-- Wind >  72.0
# |   |   |   |-- weights: [2.0, 1.0] class: Play
# |   |-- Humidity_high >  0.5
# |   |   |-- weights: [0.0, 4.0] class: dontPlay
```

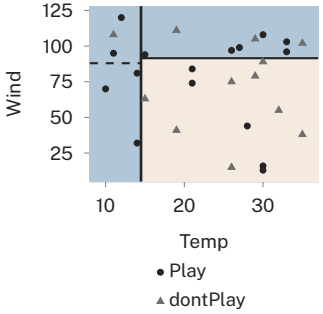
- Each `|--` line is one answer to a question, and its indentation is its depth.
- `Weather_sunny <= 0.5` is the 0 or 1 column read as "not sunny".
- A leaf prints `weights`, its counts in the order of `classes_`, which is `['Play' 'dontPlay']` because capital letters sort first.
- `[2.0, 4.0] class: dontPlay` is node 12 of R's printout. Both libraries give the new day 0.67 dontPlay.

When to Stop

Training error only falls as a tree grows, so it cannot tell you when to stop.

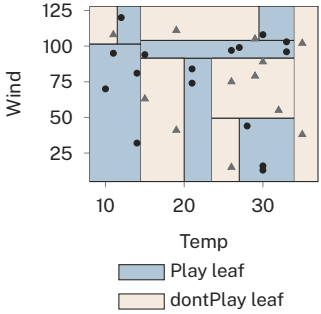
An Unstopped Tree Memorizes

two questions deep, 4 leaves



9 of 28 wrong

unstopped, 12 leaves



0 of 28 wrong

Using only `Temp` and `Wind`, no two days share a point, so some cut always separates two days of different classes. Splitting goes on until every box is pure.

The leaves hold 1, 1, 1, 1, 2, 2, 2, 2, 3, 4, 4 and 5 days. A one-day leaf copies that day's answer. Zero training error follows from the construction, so it says nothing about a 29th day.

A Smallest Leaf Size

Refuse to make a leaf with fewer than m days, the same `Temp` and `Wind` tree.

smallest leaf m	leaves	training errors
1	12	0
3	8	6
5	4	9

Python's counts. `rpart` at `cp = 0` still drops a split that lowers no training error, so it prints 12, 7 and 3 leaves. With `cp = -1` it keeps them and prints 12, 8 and 4.

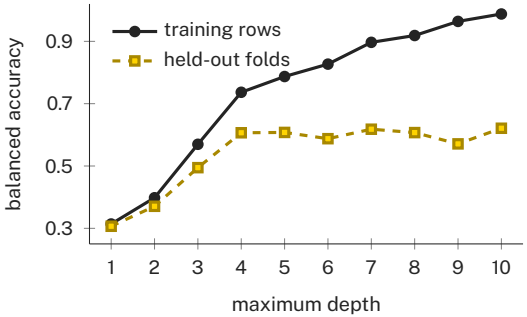
- A bigger m means fewer, larger leaves, and more training errors.
- The gain is a leaf that rests on several days, so its majority means something. Stopping early like this is called **pre-pruning**.
- The settings are `maxdepth`, `minsplit`, `minbucket` in R and `max_depth`, `min_samples_split`, `min_samples_leaf` in Python.

Training errors cannot choose m, since they always prefer $m = 1$. Choose it like K in kNN, by cross-validation.

Training Accuracy Always Rises

The weather data is too small to hold days out, so the depth is tuned on `Glass`, the six-class data the SVM was tuned on. Gini works unchanged with six shares, $1 - \sum_c p_c^2 = 0.737$ at its root.

- Training accuracy climbs to 0.99 at depth 10. The held-out folds stop near 0.61 after depth 4.
- The gap is overfitting, like 1-NN's perfect training score.
- In R, `rpart`'s training curve also climbs to 0.99, and the held-out folds R draws level off between 0.62 and 0.66.



Each point is a tree of that depth, scored on the 214 rows it learned and, refitted five times, on the fold it did not see, using Python's folds.

Choose the Depth Inside, Score Outside

R

```r
outer_score <- function(k) {
  d <- Glass[outer != k, ]
  inner <- createFolds(d$Type, 5, FALSE)
  cv <- function(depth) mean(sapply(1:5,
    function(j) {
      m <- grow(d[inner != j, ], depth)
      p <- predict(m, d[inner == j, ], "class")
      bacc(d$Type[inner == j], p)
    }))          # choose the depth INSIDE
  m <- grow(d, which.max(sapply(1:10, cv)))
  test <- Glass[outer == k, ]
  bacc(test$Type, predict(m, test, "class"))
}              # score OUTSIDE
round(sapply(1:5, outer_score), 3)
# 0.631 0.636 0.821 0.497 0.673
```

Python

```python
outer = []
for tr, te in folds.split(X, y):
    gs = GridSearchCV(                          # INSIDE
        tree.DecisionTreeClassifier(random_state=0),
        grid, cv=folds, scoring="balanced_accuracy")
    gs.fit(X[tr], y[tr])
    outer.append(balanced_accuracy_score(     # OUTSIDE
        y[te], gs.predict(X[te])))
print(np.round(outer, 3))
# [0.775 0.554 0.683 0.558 0.571]
```

The earlier nested loop with a tree in place of the SVM. R draws its own folds, and `rpart` breaks ties differently, so the two languages print different fold scores.

The Tree Beside the Other Models

Balanced accuracy on the Python listing's five outer folds of `Glass`, every model tuned inside.

Outer fold	1	2	3	4	5	mean
Baseline	0.167	0.167	0.167	0.167	0.167	0.167
LDA	0.574	0.613	0.452	0.608	0.481	0.546
kNN	0.638	0.583	0.685	0.581	0.663	0.630
SVM	0.553	0.421	0.679	0.648	0.766	0.613
Tree	0.775	0.554	0.683	0.558	0.571	0.628
Tree depth chosen	10	5	10	6	10	

The tree's mean is 0.002 below kNN's, with corrected standard error 0.057. With 4 degrees of freedom, ± 2.78 cuts off 2.5% in each tail of the t curve, as ± 2.26 did with 9, so the 95% interval is $-0.002 \pm 2.78 \times 0.057$, from -0.16 to 0.16, the gaps these folds cannot rule out. Against the SVM it is -0.30 to 0.33. Five folds cannot tell the three models apart.

The search picked depth 10 on three folds, yet the curve is flat after depth 4. Cutting a large tree back to what the evidence supports comes next.

One Day Can Change the Root

Refit 28 times, each time leaving out one day. The root question is sunny in 14 refits and overcast in 14.

Sunny beat overcast by 0.153 to 0.147, a gap of 0.006. Leaving out any of the 8 sunny dontPlay days or the 6 rain Play days swaps them, and every question below the root is then chosen on different days.

The labels move much less. Of the other 27 days, how many change label, averaged over the 28 refits?

	R	Python
tree, as fitted today	2.36	0.89
5-NN, scaled	0.93	0.93
LDA	0.46	0.50

The two tree libraries grow different trees on some 27-day samples, so their counts differ.

The rules you read change more than the labels the tree predicts. Read a fitted tree as one of several near-equal descriptions.

Trees Beside kNN, LDA and the SVM

	kNN	LDA	SVM	Tree
Needs scaling	yes	no	yes	no, only order counts
Boundary	any shape	straight	smooth curve	staircase of cuts
Can be read	no	weights	no, if radial	yes, as a path
Categorical predictors	as indicators	as indicators	as indicators	directly in `rpart`
Main setting	K	none	C, γ	depth, leaf size

Reach for a tree when the model must be read as rules or most predictors are categories. It costs a staircase boundary, a greedy search that need not find the best tree of its size, and rules that move with the sample. On `Glass`, five folds could not separate it from kNN and the SVM.

Three Ideas to Keep

DALHOUSIE UNIVERSITY

Grow

List every question, keep the one that removes the most Gini, split, and repeat inside each side.

Read

A new case follows one path to one leaf. The leaf's counts are its class shares.

Stop on evidence

Training error always falls. Choose the leaf size or depth by cross-validation, and report it.

Coming next, the same machinery for numeric targets, and pruning back to the size the evidence supports.